

Object Oriented Programming

Creational Design Patterns: Builder

IT560



Sourish Dasgupta

Assistant Professor, DA-IICT, Gandhinagar

<https://daiict.ac.in/faculty/sourish-dasgupta>

Creating Complex Objects: Building Step-by-step

```
class Animal:
    def __init__(self, species, sound, habitat, diet):
        self.species = species
        self.sound = sound
        self.habitat = habitat
        self.diet = diet

    def __str__(self):
        return (
            f"Animal: {self.species}, Sound: {self.sound}, "
            f"Habitat: {self.habitat}, Diet: {self.diet}"
        )
```

Builder Design Pattern

```
class AnimalBuilder:
    def __init__(self):
        # Default values
        self.species = None
        self.sound = None
        self.habitat = None
        self.diet = None

    def set_species(self, species):
        self.species = species
        return self

    def set_sound(self, sound):
        self.sound = sound
        return self

    def set_habitat(self, habitat):
        self.habitat = habitat
        return self

    def set_diet(self, diet):
        self.diet = diet
        return self
```

```
def build(self):
    if not self.species or not self.sound:
        raise ValueError("Species and Sound are required!")
    return Animal(self.species, self.sound, self.habitat, self.diet)
```

Builder Class (`AnimalBuilder`):

- Handles the **step-by-step** construction of an `Animal` object.
- Provides methods (`set_species`, `set_sound`, etc.) to **configure** the attributes of the `Animal` object.
- Ensures defaults or constraints can be added easily (e.g., raising an error if `species` or `sound` is missing).

Builder Design Pattern: A *detailed* Dog

```
class AnimalBuilder:
    def __init__(self):
        # Default values
        self.species = None
        self.sound = None
        self.habitat = None
        self.diet = None

    def set_species(self, species):
        self.species = species
        return self

    def set_sound(self, sound):
        self.sound = sound
        return self

    def set_habitat(self, habitat):
        self.habitat = habitat
        return self

    def set_diet(self, diet):
        self.diet = diet
        return self
```

```
def build(self):
    if not self.species or not self.sound:
        raise ValueError("Species and Sound are required!")
    return Animal(self.species, self.sound, self.habitat, self.diet)
```

```
# Usage
def main():
    # Build a Dog
    dog = (
        AnimalBuilder()
        .set_species("Dog")
        .set_sound("Woof")
        .set_habitat("Domestic")
        .set_diet("Omnivore")
        .build()
    )
    print(dog)
```

```
if __name__ == "__main__":
    main()
```

→ Chaining

Builder Design Pattern: A *minimalist* Fish

```
class AnimalBuilder:
    def __init__(self):
        # Default values
        self.species = None
        self.sound = None
        self.habitat = None
        self.diet = None

    def set_species(self, species):
        self.species = species
        return self

    def set_sound(self, sound):
        self.sound = sound
        return self

    def set_habitat(self, habitat):
        self.habitat = habitat
        return self

    def set_diet(self, diet):
        self.diet = diet
        return self
```

```
def build(self):
    if not self.species or not self.sound:
        raise ValueError("Species and Sound are required!")
    return Animal(self.species, self.sound, self.habitat, self.diet)
```

```
# Usage
def main():
    # Build a Fish with minimal attributes
    fish = (
        AnimalBuilder()
        .set_species("Fish")
        .set_sound("...")
        .set_habitat("Water")
        .build()
    )
    print(fish)
```

```
if __name__ == "__main__":
    main()
```

Why Builder Pattern?

Advantage	Explanation	Example from Code
Avoid Constructor Overload	Simplifies object creation without requiring multiple constructors for different attribute combinations.	Instead of multiple <code>Animal</code> constructors (e.g., with/without habitat, diet, etc.), <code>AnimalBuilder</code> provides a step-by-step interface.
Readable Object Creation	Object construction is easier to read and understand with named methods for setting attributes.	<code>AnimalBuilder().set_species("Dog").set_sound("Woof").build()</code> is more readable than <code>Animal("Dog", "Woof", "Domestic", "Omnivore")</code> .
Scalability	New attributes can be added easily without modifying the existing class or constructor logic.	Adding a <code>set_lifespan()</code> method in <code>AnimalBuilder</code> doesn't affect existing <code>Animal</code> code.

Why Builder Pattern?

Validation	Ensures required fields are set and invalid configurations are avoided before object creation.	The <code>build()</code> method raises an error if <code>species</code> or <code>sound</code> is missing.
Flexible Configuration	Allows objects to be constructed with varying levels of detail, from minimal to fully detailed.	Example: A minimal <code>Fish</code> with only <code>species</code> and <code>sound</code> , or a full <code>Dog</code> with habitat and diet.
Fluent Interface	Enables chaining of method calls for concise and expressive code.	<code>AnimalBuilder().set_species("Dog").set_sound("Woof").build()</code> uses method chaining for clarity.
Improved Maintainability	Separates object creation logic from the class itself, making the code easier to manage and extend.	<code>AnimalBuilder</code> encapsulates the logic for setting attributes , keeping <code>Animal</code> clean and focused.